Ders 22 Genişletilmiş Notu: Alt Dizgi Arama: Kaba Kuvvet, Boyer-Moore ve Rabin-Karp

Ders 22 Genişletilmiş Notu: Alt Dizgi Arama: Kaba Kuvvet, Boyer-Moore ve Rabin-Karp

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Alt Dizgi Arama Problemi
- 2. Kaba Kuvvet (Brute Force) Arama
- 3. Kaba Kuvvet Aramanın C Kodu
- 4. Boyer-Moore Yönteminin Fikri
- 5. Uyumsuzluk Durumları ve Kaydırma Miktarı
- 6. Kaydırma Tablosu (rightSkip)
- 7. Boyer-Moore Arama Kodu
- 8. Tam Örnek: NEEDLE Araması
- 9. Boyer-Moore Yönteminin Karmaşıklığı
- 10. Rabin-Karp Parmak İzi Yöntemi
- 11. Kayan Özet (Rolling Hash)
- 12. Üç Yöntemin Karşılaştırması

1. Alt Dizgi Arama Problemi

Hoca derse, dönem başında konuştukları amacı hatırlatarak başladı. Veri yapıları ve algoritmalar dersinin amacı, farklı uygulama alanlarında sık karşılaşılan algoritmaları tanıtmak ve bu algoritmaların kolay kullanılmasını sağlayan veri yapılarını göstermektir. Bu amaçla dönem boyunca yığın (stack), kuyruk (queue), bağlı liste (linked list), ağaç (tree) ve graf (graph) veri yapıları görüldü.

Algoritmalar tarafında en çok ihtiyaç duyulanlar arama ve sıralama algoritmalarıdır. Sıralamada daha önce kabarcık sıralaması (bubble sort) ve seçmeli sıralama (selection sort), bu dönem de araya ekleme sıralaması (insertion sort) ve yığın sıralaması (heap sort) görüldü. Hoca, böl ve yönet (divide and conquer) algoritmalarının, yani birleştirmeli sıralama (merge sort) ve hızlı sıralamanın (quicksort), sonraki yıl Algoritma Analizi dersinde işleneceğini söyledi.

Aramada şimdiye kadar tek eleman arama görüldü: bir dizide 8 var mı, 15 var mı gibi. Oysa pratikte daha sık ihtiyaç duyulan şey, **bir grup bilgiyi büyük bir bilgi içinde aramaktır**. Örneğin bir DNA dizisi içinde bir parça aramak ya da büyük bir metin içinde bir kelime aramak. Her gün kullanılan bir arama motoruna kelime yazdığımızda da benzer bir iş yapılır.

Aranan kelime, karakter, rakam ya da işaret gibi birden çok sembolden oluşan bir örüntüdür (pattern). Bu örüntüyü bir metin (text) içinde aramaya dizgi arama (string search), daha doğrusu **alt dizgi arama (substring search)** denir. “Alt” denmesinin sebebi şudur: metin N uzunluğundadır ve aranan örüntü, genelde N’den çok küçük olan M uzunluğundadır. Ana dizgi üzerinde bir alt dizgi aranır. Dersin sorusu, bu aramanın en verimli nasıl yapılacağıdır.

Ders boyunca kullanılan gösterim:

- N: metnin uzunluğu, `t[]`: metin dizisi.

- M: örüntünün uzunluğu, `p[]`: örüntü dizisi.
- `i`: örüntünün metne oturtulduğu başlangıç konumu.
- `j`: örüntü içindeki karakter indisi.

2. Kaba Kuvvet (Brute Force) Arama

Hoca örnek olarak "samanlıkta iğne aramayı" seçti: örüntü `NEEDLE` ($M = 6$), bütün harfler büyük harf. En temel yaklaşım kaba kuvvettir: bulana kadar tek tek bakılır.

İşleyiş şöyledir. Metnin ilk karakteri `N` mi diye bakılır. Değilse bir sonraki karakterden devam edilir. `N` bulunduğunda sıradaki metin karakterinin `E` olup olmadığına, sonra `E`, `D`, `L`, `E` olup olmadığına bakılır. Hepsi uyuyorsa örüntü bulunmuştur.

2.1 En İyi Durum

Hocanın tahtadaki metninde, `NEEDLE`'ın başladığı yere kadar hiç `N` harfi yoktu. Bu yüzden her konumda yalnız bir karşılaştırma yapıldı: metnin karakteri `N`'ye eşit mi, değil; bir sonraki konuma geç. `N` bulununca da kalan beş karakter karşılaştırıldı. Toplamda yaklaşık N karşılaştırma ile kelime bulundu. Bu en iyi durumdur (best case).

2.2 En Kötü Durum

Hem metinde hem örüntüde aynı harfler sürekli tekrar ederse durum kötüleşir. Hoca şu örneği verdi: metin yalnız `A`'lardan oluşup en sonda bir `B` ile bitiyor, örüntü `AAAB`.

- 1. konumda: `A=A`, `A=A`, `A=A`, sonra `B` ile `A` uymaz. 4 karşılaştırma boşa gider.
- 2. konumda (bir sağa kaydırılmış): yine üç `A` uyar, `B` uymaz. 4 karşılaştırma daha.
- Bu her konumda tekrar eder. Sonunda örüntünün `B`'si metnin `B`'sine denk geldiğinde bulunur.

Her konumda uyumsuzluk son karakterde çıktığı için her konumda M karşılaştırma yapılır. Örüntünün oturtulabileceği son konum $N - M$'dir. Hoca bunu şöyle açıkladı: `AAAB` dört harflidir; metnin sonunda üç harf kaldığında oraya artık bakmanın anlamı yoktur, çünkü kalan parça aranan kelimeden kısadır. Böylece $0, 1, \ldots, N - M$ olmak üzere $N - M + 1$ konum vardır ve en kötü durumda toplam karşılaştırma sayısı:

$$(N - M + 1) \cdot M \approx N \cdot M$$

Python ile sayıldığında, $N - 1$ tane `A` ve bir `B`'den oluşan metinde `AAAB` aranınca karşılaştırma sayısı tam olarak $(N - 4 + 1) \cdot 4$ çıkar ($N = 8$ için 20, $N = 20$ için 68).

> **Ders sonrası not:** Hoca bir yerde en kötü durumu "$(N - 1)$ çarpı M" diye söyledi. Hemen öncesinde kendisi de "$N - M + 1$ noktası" demişti; doğrusu $(N - M + 1) \cdot M$'dir.

2.3 Karmaşıklığın Yorumu

Hoca tahtaya şu özeti yazdı: en iyi durum N, en kötü durum $N \cdot M$.

- M küçükse, örneğin 100 ya da 200 ise, $N \cdot M$ en fazla $100N$ ya da $200N$ olur. Bu bizim için çok önemli bir fark değildir; karmaşıklık yine yaklaşık $O(N)$ sayılır.
- M büyükse, örneğin 1 milyon gibi değerler alıyorsa, $N \cdot M$ iyi bir değer değildir. M büyüdükçe karmaşıklık $O(N^2)$'ye gidebilir.

Sonuç olarak kaba kuvvet, kısa alt dizgiler için kullanılmasında sakınca olmayan, ama uzun alt dizgilerde yavaş çalışmaya sebep olabilecek bir yöntemdir.

3. Kaba Kuvvet Aramanın C Kodu

Fonksiyon iki parametre alır: aranan örüntü `p` ve üzerinde arama yapılacak metin `t`. Önce iki uzunluk hesaplanır: `n` metnin, `m` örüntünün uzunluğudur.

Dış döngü, örüntünün metne oturtulduğu konumu `i` ile gezer. Yukarıda anlatıldığı gibi son bakılacak konum `n - m` olduğu için döngü `i <= n - m` sürece döner.

İç döngü her konumda `j = 0`'dan başlar ve iki koşul sağlandığı sürece `j`'yi artırır:

- `j < m`: örüntünün bütün karakterlerine henüz bakılmamıştır.
- `p[j] == t[i + j]`: örüntünün `j`'inci karakteri metnin `i + j`'inci karakterine eşittir.

Hoca ikinci koşulu bir örnekle açıkladı. $i = 3$ olsun. $j = 0$ için metnin 3. gözü örüntünün 0. gözüyle, $j = 1$ için metnin 4. gözü örüntünün 1. gözüyle, $j = 2$ için metnin 5. gözü örüntünün 2. gözüyle karşılaştırılır.

İç döngüden iki sebeple çıkılır. `j == m` ise örüntünün bütün karakterleri tek tek uymuştur, yani örüntü bulunmuştur ve `i` döndürülür. Değilse bir yerde eşitlik bozulmuştur; `i` bir artırılır ve bir sonraki konumdan aramaya devam edilir. Dış döngü hiç `return` olmadan biterse örüntü yoktur ve `-1` döndürülür.

```
int search(char p[], char t[])
{
    int i, j, n, m;
    n = strlen(t);
    m = strlen(p);
    for (i = 0; i <= n - m; i++) {
        j = 0;
        while ((j < m) && (p[j] == t[i + j]))
            j++;
        if (j == m)
            return i;       /* bulundu: örüntünün metindeki başlangıcı */
    }
    return -1;              /* bulunamadı */
}
```

3.1 `return` ile Döngüden Çıkmak

Soru: `return i` dendiğinde bu `break` gibi davranıp `for` döngüsünü kırıp çıkmıyor mu?

Cevap: Hoca bunun döngü kırmak gibi düşünülmemesi gerektiğini, burada fonksiyondan çıkıldığını söyledi. Soruyu güzel bulup aynı algoritmayı `while` ve bir `found` bayrağıyla yeniden yazdı. `for` yerine `i = 0` ile başlanır, döngü `i <= n - m` ve `!found` sürece döner; `i++` döngünün içine, `else` dalına taşınır:

```
int search2(char p[], char t[])
{
    int i, j, n, m, found;
    n = strlen(t);
    m = strlen(p);
    i = 0; found = 0;
    while ((i <= n - m) && (!found)) {
        j = 0;
        while ((j < m) && (p[j] == t[i + j]))
            j++;
        if (j == m)
            found = 1;
        else
            i++;
    }
    if (found == 1)
        return i;
    else
        return -1;
}
```

Hoca ilk sürümü `i`'nin 0'dan `n - m`'ye kadar birer birer arttığını açıkça görmek için `for` ile yazdığını söyledi. Fakat `return` ile aradan çıkmanın bir tehlikesi vardır: büyük bir program

parçasında okuyan kişi fonksiyondan nerede ve nasıl çıkıldığını anlamakta güçlük çekebilir. `while` ile yazıldığında çıkış koşulu döngünün başında açıkça görünür.

> **Hocanın vurgusu:** Düz bir program içinde döngüleri `break` ile kırmayı asla yapmayın. Fonksiyondan `return` ile dönmek o kadar sakıncalı değildir; yalnız okunabilirlik açısından sorun yaratır.

Her iki fonksiyon `gcc -Wall` ile uyarısız derlendi. `FINDINAHAYSTACKNEEDLEINA` metninde `NEEDLE` için ikisi de 15, 7 `A` ve 1 `B`'den oluşan metinde `AAAB` için ikisi de 4 döndürdü (tam program 7. bölümde).

3.2 Kaba Kuvvetin Sorunu

Hoca sorunu şöyle özetledi: sona kadar karşılaştıra karşılaştıra gelinir, son karakter uymaz, sonra başa dönülüp bir sonraki karakterden yeniden başlanır. Arada bir yerde `AAAB` çıkma ihtimaline karşı, emin olmak için fazladan fazladan karşılaştırma yapılır. Metindeki aynı karakterlerle defalarca karşılaştırma yapılır. Peki bu problemi başka bir şekilde çözmek mümkün mü?

4. Boyer-Moore Yönteminin Fikri

Bunun için kullanılan yöntemlerden biri **Boyer-Moore yöntemidir**. Örüntü yine metnin başına oturtulur, ama karşılaştırma **örüntünün sonundan başına doğru** yapılır.

Hoca fikri `NEEDLE` örneğiyle anlattı. Örüntü metnin başına oturtulduğunda önce örüntünün son karakteri `E` ile metnin o konumdaki karakteri karşılaştırılır. Burada metinde `N` vardır ve `E` ile uymaz.

Şimdi bir tehlike vardır. Uyumsuzluk görünce örüntü uzunluğu kadar birden atlanırsa, metinde arada bir yerde duran bir `NEEDLE` kaçırılabilir. Boyer-Moore bu tehlikeden şöyle kurtulur: uyumsuz karakter `N`'dir; `NEEDLE` içinde `N` var mı? Var. O halde örüntü, iki `N` çakışacak şekilde kaydırılır. Örüntü uzunluğu kadar körlemesine atlamak yerine, uyumsuz karakter örüntüdeki aynı karakterle hizalanır.

Sonraki konumda son karakter `E`, metindeki `S` ile karşılaştırılır ve uymaz. `NEEDLE` içinde `S` var mı? Yok. O zaman gönül rahatlığıyla `S`'nin tamamen ötesine atlanır, çünkü `S`'yi içeren hiçbir hizalamada `NEEDLE` bulunamaz.

Sonraki konumda `E=E` uyar, ardından `N` ile `L` uymaz. Yine `NEEDLE` içinde `N` aranır ve `N`'ler çakışacak şekilde kaydırılır. Bu konumda sondan başa doğru `E`, `L`, `D`, `E`, `E`, `N` hepsi uyar ve örüntü bulunur.

Hoca iki yöntemin farkını şöyle bağladı: kaba kuvvette, örüntü arada bir yerde geçerse kaybedilmesin diye her uyumsuzlukta yalnız bir karakter ilerlenir ve metnin aynı karakterlerine defalarca bakılır. Boyer-Moore bunu önlemek ister. Uyumsuz karaktere bakar, örüntüde var mı diye kontrol eder ve ona göre kaydırır.

Soru: Uyumsuz karakter örüntü içinde birkaç defa geçiyorsa (örneğin `NEEDLE`'da üç tane `E` var) hangisiyle çakıştıracağız?

Cevap: Hoca soruyu aradan sonraya bıraktı ve aradan sonra durumları anlatırken cevapladı: karakterin örüntüdeki **en sağdaki (rightmost) yeri** ile çakıştırılır. Böylece hiçbir hizalama atlanmaz ve kelime hiçbir zaman kaçırılmaz.

5. Uyumsuzluk Durumları ve Kaydırma Miktarı

Aradan sonra hoca, uyumsuzluk (mismatch) olduğunda ne kadar kayılacağına nasıl karar verileceğini sistematik olarak anlattı. Temel akış şudur: `i` 0'dan başlar. Her konumda `j`, $M-1$'den başlatılır, çünkü sondan başa gidilir. `j >= 0` ve `p[j] == t[i + j]` olduğu sürece `j` azaltılır. Döngüden çıkıldığında `j >= 0` ise örüntü bu konumda bulunamamıştır ve bir kaydırma (skip) hesaplanmalıdır.

> **Hocanın vurgusu:** Algoritma tasarlarken hep böyle yapmalısınız. Önce küçük bir bilgi parçası üzerinde analiz yaparak hangi şartları göz önünde bulundurmanız gerektiğine bakın.

Hoca önce akışı küçük bir örnekle gösterdi: `SOPA` ($M = 4$) örüntüsü metinde $i = 5$ konumuna oturtulmuş olsun. $j = 3$'te metnin $5+3 = 8$. karakteri örüntünün 3. karakteriyle karşılaştırılır ve uyar. $j = 2$'de metnin 7. karakteri örüntünün 2. karakteriyle karşılaştırılır ve uyar. $j = 1$'de metnin 6. karakteri ile örüntünün 1. karakteri karşılaştırılır ve eşitlik bozulur. Bütün mesele bu bozulma durumunu anlamaktır.

5.1 Durum 1: Uyumsuz Karakter Örüntüde Yok

En kolay durumdur. Hoca örnekte metni `...KÖPAK...` olarak aldı: metnin 5-9. karakterleri `K`, `Ö`, `P`, `A`, `K` olsun. `SOPA` $i = 5$'e oturtulduğunda `A=A` ($j = 3$), `P=P` ($j = 2$) uyar, $j = 1$'de metindeki `Ö` ile örüntüdeki `O` uymaz.

`Ö` harfi `SOPA` içinde yoktur. Öyleyse `Ö`'yü kapsayan hiçbir hizalamada `SOPA` bulunamaz; yola `Ö`'den sonrasından devam edilmelidir. Yeni konum:

$$i = i + j + 1 = 5 + 1 + 1 = 7$$

Böylece arama metnin 7. karakterinden, yani `PAK`'tan itibaren sürer.

Hoca önce metni `KAPAK` olarak yazmış, sonra orada `A` bulunduğu için (`A` örüntüde var, bu ikinci duruma girer) örneği `KÖPAK` olarak değiştirdi.

5.2 Durum 2a: Uyumsuz Karakter Örüntüde Var, Kaydırma Pozitif

Hoca bu durumu iki örnekle gösterdi.

Birinci örnek (`NEEDLE`). $i = 10$ olsun. $j = 5$'te örüntünün son `E`'si ile metindeki `N` uymaz. `N` örüntüde vardır ve örüntünün 0. karakteridir. `N`'ler çakışacak şekilde oturtmak için:

$$i = i + j - 0 = 10 + 5 - 0 = 15$$

İkinci örnek (`KARA`). Hoca bu örneği, ilk denemelerinde tahtada karışıklık olunca yeniden kurdu. Metin `...SOKARA...` olsun: `S` 9., `O` 10., `K` 11., `A` 12., `R` 13., `A` 14. konumda. Örüntü `KARA` ($M = 4$), $i = 9$'a oturtulmuş, `j` değerleri 3, 2, 1, 0 diye gidecek.

- $j = 3$: metnin 12. karakteri `A`, örüntünün 3. karakteri `A`. Uyar.
- $j = 2$: metnin 11. karakteri `K`, örüntünün 2. karakteri `R`. Uymaz.
- `KARA` kelimesini kaçırmamak için örüntüde `K` var mı diye bakılır. Var, en sağdaki yeri 0. göz.
- Yeni konum: $i = i + j - 0 = 9 + 2 - 0 = 11$.

$i = 11$ olunca örüntünün `K`'sı metnin 11. karakterindeki `K`'ya denk gelir ve metnin 14. karakteri örüntünün 3. karakteriyle karşılaştırılır. Bu konumda `KARA` bulunur.

Genel kural, `rightmost(c)` uyumsuz karakterin örüntüdeki en sağdaki indisi olmak üzere:

$$i = i + j - \text{rightmost}(c)$$

5.3 Durum 2b: Uyumsuz Karakter Örüntüde Var, Kaydırma Negatif

En sağdaki konuma göre hizalamak bazen örüntüyü geriye kaydırır. Geriye gidilirse daha önce bakılmış yerlere dönülür; oradan gelindiği ve orada örüntünün olmadığı zaten bilinmektedir. Bu durumda yapılacak şey yalnız $i = i + 1$ deyip yola devam etmektir.

Hocanın örneği: örüntü `NEEDLE`, $i = 10$. $j = 5$ ve $j = 4$'te karakterler uyar, $j = 3$'te bozulur ve metindeki uyumsuz karakter `E`'dir. `E`'nin `NEEDLE`'daki en sağdaki yeri 5'tir. Kurala göre:

$$i + j - 5 = 10 + 3 - 5 = 8$$

$8 < 10$ olduğu için örüntü geri kayacaktır. Geri kaymanın anlamı olmadığından $i = 11$ yapılır.

> **Ders sonrası not:** Hoca bu örnekte önce i'nin 6 olacağını, sonra "$10 + 1 - 5$" gibi değerler söyledi. Bir öğrencinin uyarısıyla düzeltip doğru hesabı $10+3-5 = 8$ olarak verdi. Ayrıca hoca durum 2a'yı özetlerken bir yerde "en soldaki yer" dedi; hemen öncesinde ve sonrasında doğru olarak "en sağdaki (son) yer" dediği için kastedilen en sağdaki yerdir.

5.4 Durumların Özeti

Hoca üç durumu tahtada birlikte yazdı. Aslında iki ana durum vardır: karakter örüntüde yoktur ya da vardır. İkincisi de kaydırmanın işaretine göre ikiye ayrılır.

Durum	Koşul	Yeni `i`
1	Uyumsuz karakter örüntüde yok	`i = i + j + 1`
2a	Karakter örüntüde var, kaydırma pozitif	`i = i + j - rightmost`
2b	Karakter örüntüde var, kaydırma negatif	`i = i + 1`

> **Ders sonrası not:** Hoca durum 1'i önce `i = i + j - 1` diye yazdı. Birkaç dakika sonra yanlış yazdığını söyleyip `i = i + j + 1` olarak düzeltti. Doğrusu `i + j + 1`'dir.

6. Kaydırma Tablosu (rightSkip)

Durumlar belli olunca hoca bir şeye dikkat çekti: her uyumsuzlukta "örüntü içinde `D` var mı, `S` var mı, `N` var mı?" diye örüntüyü tekrar tekrar taramak gerekiyor. Oysa **örüntü hiçbir zaman değişmez**. `S` ile `E` çakışmadığında ne kadar atlanacağı da, `L` ile `N` çakışmadığında `N`'nin konumuna göre ne kadar atlanacağı da baştan bellidir. O halde bu kontroller her seferinde yapılmadan, bir arama tablosu (look-up table) üzerinden sağlanabilir.

> **Hocanın vurgusu:** Bu algoritmanın asıl güzelliği budur: örüntü değişmediği için kaydırma bilgisi önceden bir tabloya yazılır ve her uyumsuzlukta yalnız tabloya bakılır.

Hoca, kaydırma miktarını bu şekilde tabloyla bulma fikrini Nigel Horspool'a bağladı ve 1980'lerde geliştirildiğini söyledi.

6.1 Tablonun İçeriği

Alfabe yalnız 26 büyük harften oluşsun. Tabloda her harf için bir göz vardır:

- Örüntüde geçen her harfin gözüne, o harfin örüntüdeki **en sağdaki indisi** yazılır.
- Örüntüde geçmeyen bütün harflerin gözüne -1 yazılır.

`NEEDLE` için (indisler 0-5: `N`, `E`, `E`, `D`, `L`, `E`):

Karakter	N	E	D	L	Diğerleri
`rightSkip`	0	5	3	4	-1

Bu tabloyla kaydırma tek formüle iner:

$$\text{skip} = j - \text{rightSkip}[t[i + j]]$$

Örüntüde olmayan karakter için tablo değeri -1 olduğundan skip $= j - (-1) = j + 1$ olur; bu tam olarak durum 1'dir. Örüntüde olan karakter için durum 2a'nın formülü çıkar. Sonuç negatifse durum 2b gereği `skip = 1` alınır.

Hoca bunu KEDI örneğiyle de gösterdi. Metinde KASAP yazıyor, KEDI $i = 9$'a oturtulmuş, $j = 3$'te metindeki S ile örüntünün I'sı uymuyor. S, KEDI içinde yoktur; istenen, S'nin ötesine atlamaktır: $i = 9 + 3 + 1 = 13$. Tablodan: skip $= 3 - (-1) = 4$, $i = 9 + 4 = 13$.

> **Ders sonrası not:** Hoca bu örnekte önce `skip = j - lookup[S] - 1` gibi bir ifadeyle 2 ve $i = 11$ buldu, sonra "j eksi eksi 1, j artı 1, 4" diyerek düzeltip $i = 13$'e ulaştı. Doğru formül skip $= j -$ rightSkip$[c]$'dir, ayrıca -1 çıkarılmaz.

6.2 Tabloyu Kuran Fonksiyon ve offset

Fonksiyon önce örüntünün uzunluğunu alır. Tablo boyutu `tableN`, arama hangi karakterler üzerinde yapılıyorsa ona göre seçilir; büyük harfler için 26'dır. Önce tablonun bütün gözlerine -1 yazılır. Sonra örüntü soldan sağa gezilir ve her karakterin gözüne indisi yazılır.

`offset` parametresi, karakteri tablo indisine çevirmek için kullanılır. Tablo büyük harfler içinse `offset = 'A'`'dır: `'B' - 'A' = 1` olduğu için B tablonun 1. gözüne, `'A' - 'A' = 0` olduğu için A 0. gözüne düşer. Örneğin NEEDLE'ın 0. gözünde N olduğu için tablonun N gözüne 0, 4. gözünde L olduğu için L gözüne 4 yazılır.

```
void shiftLookUp(char pattern[], int rightSkip[], int offset, int tableN)
{
    int i, m = strlen(pattern);
    for (i = 0; i < tableN; i++)
        rightSkip[i] = -1;
    for (i = 0; i < m; i++)
        rightSkip[pattern[i] - offset] = i;
}
```

Aynı karakter örüntüde birden çok kez geçiyorsa, soldan sağa gezildiği için sonraki yazım öncekini ezer. Böylece tabloda karakterin en sağdaki indisi kalır. NEEDLE'da E için önce 1, sonra 2, en son 5 yazılır.

> **Ders sonrası not:** Hoca ilk döngüyü anlatırken "başlangıçta tablonun tamamını sıfırlıyorum" dedi ama tahtaya `rightSkip[i] = -1` yazdı. Doğrusu -1'dir; 0 yazılırsa örüntüde olmayan karakterler örüntünün 0. gözünde varmış gibi görünür.

Soru: Bu offset her zaman A mı oluyor?

Cevap: Hayır, offset bu yüzden parametredir. Tablo büyük harflerden oluşuyorsa offset `'A'`, küçük harflerden oluşuyorsa `'a'` olur. Başka sembollerden oluşuyorsa ona göre seçilir. Bütün ASCII tablosu kullanılacaksa offset hiç kullanılmadan karakter kodları doğrudan indis olarak kullanılabilir.

7. Boyer-Moore Arama Kodu

Hoca arama fonksiyonunu şu koşullarla kurdu:

- Dış döngünün iki şartı vardır: son bakılabilecek konuma kadar ilerlemek (`i <= N - M`) ve durma şartının sağlanmamış olması (`!stop`).
- Her konumda `j = M - 1`'den başlanır, çünkü sondan başa gidilir.
- İç döngü `j >= 0` ve `p[j] == t[i + j]` olduğu sürece `j`'yi azaltır.
- İç döngüden çıkıldığında `j >= 0` ise eşitlik bir yerde bozulmuştur; kaydırma hesaplanır, negatifse 1 yapılır ve `i` o kadar artırılır.
- `j == -1` ise bütün harflere bakılmış ve örüntü bulunmuştur; `stop = 1` yapılır.

Hoca iç döngünün koşulunu anlatırken önce "`j == -1` ise bulunamadı" dedi. Bir öğrenci "bulundu olması gerekmiyor mu?" diye sordu; hoca haklı olduğunu söyleyip düzeltti: `j == -1` ise bütün harflere bakılmış ve örüntü bulunmuştur, `j >= 0` ise aramaya devam edilmektedir.

Aşağıdaki program dersteki üç fonksiyonu bir arada içerir ve `gcc -Wall` ile uyarısız derlenip çalıştırıldı. Tablo `offset` ile kurulduğu için arama fonksiyonunda tabloya `t[i + j] - OFFSET` indisiyle bakılır (aşağıdaki ders sonrası nota bakın).

```
#include <stdio.h>
#include <string.h>
```

```c
#define TABLEN 26      /* yalnız büyük harfler */
#define OFFSET 'A'

int rightSkip[TABLEN];

void shiftLookUp(char pattern[], int rightSkip[], int offset, int tableN)
{
    int i, m = strlen(pattern);
    for (i = 0; i < tableN; i++)
        rightSkip[i] = -1;
    for (i = 0; i < m; i++)
        rightSkip[pattern[i] - offset] = i;
}

int subStringSearch(char p[], char t[])   /* p: aranan, t: içinde arama yapılan */
{
    int i, j, skip = 0, N = strlen(t), M = strlen(p), stop = 0;
    i = 0;
    while ((i <= N - M) && (!stop)) {
        j = M - 1;
        while ((j >= 0) && (p[j] == t[i + j]))
            j--;
        if (j >= 0) {                     /* bu konumda bulunamadı, kaydır */
            skip = j - rightSkip[t[i + j] - OFFSET];
            if (skip < 0)
                skip = 1;
            i += skip;
        }
        else                              /* j == -1: bulundu */
            stop = 1;
    }
    if (stop == 1)
        return i;
    else
        return -1;
}

int main(void)
{
    char t[] = "FINDINAHAYSTACKNEEDLEINA";
    char p[] = "NEEDLE";
    shiftLookUp(p, rightSkip, OFFSET, TABLEN);
    printf("%d\n", subStringSearch(p, t));   /* 15 yazar */
    return 0;
}
```

Ders sonrası not: Tahtadaki arama kodunda tabloya `rightSkip[t[i + j]]` diye bakıldı. Tablo `offset` ile kurulduğu için burada da `rightSkip[t[i + j] - offset]` kullanılmalıdır; aksi halde `'A'` için 65. göze bakılır ve 26 boyutlu dizinin dışına çıkılır. Hoca örnek üzerinde konuşurken "right skip'in `N` eksi `A`'ıncı gözü" diyerek bu indisi doğru kullandı.

Ek bilgi (derste anlatılmadı): Uyumsuzluk anında `skip` hiçbir zaman 0 çıkamaz. `skip = 0` olması için uyumsuz karakterin en sağdaki indisinin tam `j` olması gerekir; bu da `p[j]`'nin o karakter olduğu, yani karakterlerin uyduğu anlamına gelir. Bu yüzden koddaki `skip < 0` kontrolü ile kısa notta geçen `skip < 1` kontrolü aynı sonucu verir. Bu, A ve B harflerinden oluşan 20000 rastgele örüntü-metin çiftiyle python'da denendi; `skip` hiç 0 çıkmadı ve sonuç her seferinde doğrudan aramayla aynıydı.

8. Tam Örnek: NEEDLE Araması

Hoca bölümü, kaydırma kısmının üzerinden bir kez daha geçerek ve bir örneği yavaşça adım adım çözerek bitirdi. Örüntü `NEEDLE`, metin `FINDINAHAYSTACKNEEDLEINA` (indisler 0'dan başlar). Tablo: `N` = 0, `E` = 5, `D` = 3, `L` = 4, diğerleri -1.

> **Ders sonrası not:** Hoca bu örneğe başlarken "m eşittir 5" dedi. `NEEDLE` altı harflidir, $M = 6$'dır; hoca derste de j'yi 5'ten başlattığı için hesaplar $M = 6$ ile yapılmıştır.

Adım 1: $i = 0$**.** $j = 5$'te metnin $0 + 5 = 5$. karakteri `N`, örüntünün `E`'si ile uymaz. `N` için tablo değeri, yani `rightSkip['N' - 'A']`, 0'dır.

$$\text{skip} = 5 - 0 = 5, \quad i = 0 + 5 = 5$$

`NEEDLE`, `N`'ler çakışacak şekilde oturur.

Adım 2: $i = 5$**.** $j = 5$'te metnin 10. karakteri `S`, `E` ile uymaz. `S` örüntüde yoktur, tablo değeri -1.

$$\text{skip} = 5 - (-1) = 6, \quad i = 5 + 6 = 11$$

Adım 3: $i = 11$**.** $j = 5$'te metnin 16. karakteri `E`, `E` ile uyar. $j = 4$'te metnin 15. karakteri `N`, örüntünün `L`'si ile uymaz. `N` için tablo değeri 0.

$$\text{skip} = 4 - 0 = 4, \quad i = 11 + 4 = 15$$

Adım 4: $i = 15$**.** Sondan başa `E=E`, `L=L`, `D=D`, `E=E`, `E=E`, `N=N`. $j = -1$ olur; örüntü 15. konumda bulunur.

Adım	`i`	Uyumsuz karşılaştırma	Tablo değeri	`skip`	Yeni `i`
1	0	`t[5]` = `N`, `p[5]` = `E`	0	5	5
2	5	`t[10]` = `S`, `p[5]` = `E`	-1	6	11
3	11	`t[15]` = `N`, `p[4]` = `L`	0	4	15
4	15	Bütün karakterler uyar	-	-	Bulundu

Bu iz, yukarıdaki C programına eklenen `printf` ile birebir aynı çıktı: `i` sırasıyla 0, 5, 11, 15 ve sonuç 15. Toplam karakter karşılaştırması 10'dur; aynı metinde kaba kuvvet 23 karşılaştırma yapar (python ile sayıldı).

8.1 Öğrencinin Önerdiği Değişik Durum

Soru: Bir öğrenci, metindeki ardışık iki karakterin (örüntünün 4. ve 5. gözlerine denk gelenlerin) ikisi de `E` olsaydı ne olacağını sordu.

Cevap: Hoca durumu tahtada kurdu: $i = 9$, metnin 10-14. konumları gidiyor, $j = 5$'te `E=E` uyuyor, $j = 4$'te metindeki `E` örüntünün `L`'si ile uymuyor. Kural $i = i + j -$ rightSkip$[c]$'dir. Uyumsuz karakter `E`'dir ve tablodaki değeri (`rightSkip['E' - 'A']`) 5'tir.

$$i + j - 5 = 9 + 4 - 5 = 8$$

i, 9'dan 8'e geri düşmüştür, yani kaydırma negatiftir ($4 - 5 = -1$). Bu durumda i yalnız bir artırılır ve aramaya $i = 10$'dan devam edilir. Bu, durum 2b'nin örneğidir.

9. Boyer-Moore Yönteminin Karmaşıklığı

Hoca, çok kötü olmayan bir metinde Boyer-Moore'un ortalama olarak yaklaşık N/M karşılaştırma yaptığını söyledi. Sebebi şudur: uyumsuzluk çıktığı anda tabloya bakılır ve gerektiği kadar kaydırılır. Normal bir metinde ortalama bir iki karakter karşılaştırılır, uyumsuzluk çıkar, kaydırılır; çoğu zaman neredeyse bir örüntü boyu atlanır.

En kötü durumu ise sürekli tekrar eden karakterler doğurur. Hoca örnekte metni `ABBB-BABBBB...` gibi `B`'lerin çok tekrar ettiği bir dizgi, örüntüyü `ABBBB` olarak verdi. Örüntü bir yere oturtulur, sondan geriye karşılaştırılır; `B`'ler hep uyar ve uyumsuzluk en baştaki harfte çıkar. Neredeyse M karakter karşılaştırılmış olur. Örüntü tekrar konur, yine M karşılaştırma yapılır ve en başta bozulur. Bu durumda algoritma ne kadar iyi olursa olsun en kötü durum $N \cdot M$ olur.

Python ile denendiğinde, yalnız `B`'lerden oluşan $N = 20$ uzunluklu metinde `ABBBB` aranınca her konumda 5 karşılaştırma yapılıp `i` yalnız 1 artar; toplam $5 \cdot 16 = 80$ karşılaştırma çıkar ($N = 100$ için 480).

> **Hocanın vurgusu:** Düzgün, anlamlı bir metinde bu kadar çok tekrar eden bir yapı olasılığı çok büyük değildir. Uyumsuzluk genelde erken çıktığı için Boyer-Moore pratikte yaklaşık N/M karşılaştırma yapar.

10. Rabin-Karp Parmak İzi Yöntemi

Hoca, alt dizgi arama için çok sayıda yöntem olduğunu, derste yalnız basit bir iki tanesinin işlendiğini söyleyerek son yöntemi anlattı: **Rabin-Karp parmak izi (fingerprint) yöntemi**.

Fikir şudur: harfler tek tek karşılaştırılmaz. Örüntüden bir sayı üretilir ve bu sayı metindeki her M uzunluklu parçanın sayısıyla bir bütün olarak karşılaştırılır. Boyer-Moore'da bile sondan başa gidip karakter karakter bakmak gerekiyordu. Rabin-Karp ise baştan bir hazırlık yapar ve karşılaştırmayı tek bir sayı üzerinden yapar.

10.1 Kelimeden Sayı Üretmek

Yalnız büyük harflerden oluşan bir metinde harflere sırasıyla sayı karşılığı verilsin: `A` = 0, `B` = 1, `C` = 2 gibi. Bu durumda `BACA` kelimesi 1, 0, 2, 0 basamaklarından oluşan **1020** sayısıyla gösterilebilir.

Hoca örnek metni `KARABACA` gibi yazdı ve dört karakterlik parçaların sayı karşılıklarını (`KARA`, `ARAB`, `RABA`, …) uydurma değerlerle gösterdi (kendisi de bu değerleri "tamamen atıyorum" diye belirtti). Her parçanın sayısı 1020 ile karşılaştırılır; `BACA`'ya gelindiğinde 1020 bulunur ve arama biter.

> **Ders sonrası not:** Hoca bu ilk anlatımda pencereyi "4 karakter atlayarak" ilerlettiğini söyledi. Yöntemde pencere her adımda bir karakter kayar; aksi halde dört karakterlik sınırlara denk gelmeyen bir eşleşme kaçırılır. Hoca hemen ardından gösterdiği sayısal örnekte bunu doğru olarak "birer birer kaydıra kaydıra gidiyor" diye anlattı.

10.2 Büyük Sayılar ve Mod Alma

Uzun örüntülerde üretilen sayı çok büyür. Bu yüzden sayının, **büyük bir asal sayı** Q'ya göre modu alınır ve karşılaştırma bu kalanlarla yapılır.

Hocanın gösterdiği örnek (rakamlardan oluşan metin, $M = 5$, $Q = 997$):

- Örüntü: `26535`. Özeti: $26535 \bmod 997 = 613$.
- Metin: `3141592653589793`.

Her adımda pencere bir karakter sağa kayar, pencerenin sayısının 997'ye göre modu alınır ve 613 ile karşılaştırılır. Karşılaştırma basamak basamak (3 mü, 1 mi, 4 mü…) yapılmaz; 14159 gibi bir sayı bütün olarak karşılaştırılır.

i	Pencere	mod997	613 ile
0	31415	508	farklı
1	14159	201	farklı
2	41592	715	farklı
3	15926	971	farklı
4	59265	442	farklı
5	92653	929	farklı
6	26535	613	eşleşme

Ara adım örneği: $997 \cdot 31 = 30907$ ve $31415 - 30907 = 508$ (ara adım eklendi). Bütün değerler python ile hesaplandı. Ayrıca bu örnek, kayan özetle çalışan bir C programıyla (`gcc -Wall`, uyarısız) çalıştırıldı; pencere özetleri tablodakiyle aynı çıktı ve sonuç $i = 6$ oldu.

> **Ders sonrası not:** Hoca ilk pencere için "31415'in 997'ye göre modu 518" dedi. Doğrusu 508'dir. Hoca ikinci pencereyi de bir yerde "14179" diye okudu; pencere 14159'dur ve modu, hocanın da söylediği gibi 201'dir.

10.3 Çakışma (Collision)

Soru: Hatalı sonuç olma ihtimali yok mu? Örneğin bir sayı 997'nin 2 katının 613 fazlası, bir başkası 3 katının 613 fazlası olabilir; ikisinin de modu 613 çıkar.

Cevap: Hoca soruyu çok güzel buldu ve önce yöntemi soyut bir x sayısıyla tekrar anlattı: aranan sayı 41592 olsun ve x'e göre modu 579 olsun. 31415'in modu 561 çıksın, eşit değil; 14159'unki 375 çıksın, eşit değil; 41592'ye gelinince 579 bulunur. Fakat bambaşka bir sayının, örneğin 75356'nın da x'e göre modu 579 olabilir. Bu yüzden **özetler eşit çıktığında karakterler tek tek karşılaştırılır** (4 mü, 1 mi, 5 mi, 9 mu, 2 mi). Eşitse örüntü bulunmuştur. Bu tek tek karşılaştırma yalnız özetlerin eşit çıktığı konumlarda yapılır.

Çakışmayı azaltmak için Q mutlaka büyük bir asal sayı seçilir. Hoca bunu şöyle açıkladı: örneğin $Q = 100$ alınsaydı 100, 200, 300, … gibi katlar ve ortak çarpanlar yüzünden modu aynı çıkan değerler çok olurdu. Asal sayı bu ihtimali azaltır, ama yine de çakışma mutlaka olabilir.

> **Hocanın vurgusu:** Farklı sayıların modu aynı çıkabilir. Özet eşitliği bulunduğu anlamına gelmez; eşitlik görüldüğü anda karakter karakter karşılaştırma yapılmalıdır.

Soru: Karakter karakter karşılaştırmak yerine, bölmede çıkan bölümler de saklanıp karşılaştırılsa olmaz mı? (Örneğin 26535'i 997'ye böldüğümüzde çıkan bölüm.)

Cevap: Hoca önerinin doğru olduğunu kabul etti: kalanları aynı olan iki sayının bölümleri farklıysa (biri 997'nin 5 katından, diğeri 9 katından kalan gibi), bölümlerle de ayırt edilebilir. Ancak Rabin-Karp yönteminde bu noktada karakterler tek tek karşılaştırılır.

11. Kayan Özet (Rolling Hash)

Hoca sınıfa bir soru yöneltti: 3, 1, 4, 1, 5 ayrı basamaklardır. Önce bunlardan 31415 sayısı oluşturulur, sonra modu alınır. Ardından 1, 4, 1, 5, 9'dan 14159 oluşturulur ve modu alınır. Sonra 41592… Her seferinde sayı baştan oluşturuluyor. **Bu nasıl kısaltılabilir?**

Bir öğrenci, daha önce bu algoritmayı gördüğünü söyleyerek cevap verdi: baştaki 3 basamağının katkısı (30000) çıkarılır, kalan katsayıyla (tabanla) çarpılır ve sağdan gelen yeni basamak eklenir. Hoca cevabı onaylayıp formülünü kurdu.

11.1 Pencere Sayısının Formülü

B taban olmak üzere (bu örnekte $B = 10$), i'inci pencerenin sayısı:

$$x_i = t_i \cdot B^{M-1} + t_{i+1} \cdot B^{M-2} + \cdots + t_{i+M-1} \cdot B^0$$

Hoca bunu 31415 için açtı ($M = 5$):

$$31415 = 3 \cdot 10^4 + 1 \cdot 10^3 + 4 \cdot 10^2 + 1 \cdot 10^1 + 5 \cdot 10^0$$

M basamaklı bir sayı B tabanında bu şekilde bir döngüyle oluşturulur.

11.2 Bir Önceki Pencereden Yararlanmak

Bir sonraki pencere:

$$x_{i+1} = t_{i+1} \cdot B^{M-1} + t_{i+2} \cdot B^{M-2} + \cdots + t_{i+M} \cdot B^0$$

İki ifade yan yana konunca x_{i+1}'de baştaki $t_i \cdot B^{M-1}$ terimi yoktur, sonda ise $t_{i+M} \cdot B^0 = t_{i+M}$ terimi fazladır. Yani baştaki basamak çıkarılır, sondaki basamak eklenir. Kalan basamakların her biri de bir basamak sola kaydığı için B ile çarpılmalıdır:

$$x_{i+1} = (x_i - t_i \cdot B^{M-1}) \cdot B + t_{i+M}$$

Örnek ($B = 10$, $M = 5$):

$$31415 \ \rightarrow \ (31415 - 3 \cdot 10^4) \cdot 10 + 9 = 1415 \cdot 10 + 9 = 14159$$

Hoca, her seferinde bu hesabı boşu boşuna baştan yapmak yerine bir önceki bilgiden yararlanmanın son derece verimli olduğunu vurguladı.

> **Ders sonrası not:** Tahtadaki formül $x_{i+1} = x_i - t_i \cdot B^{M-1} + t_{i+M}$ şeklinde yazıldı. Baştaki basamak çıkarıldıktan sonra kalan kısım B ile çarpılmalıdır. Çarpım olmadan $31415 - 30000 + 9 = 1424$ çıkar, 14159 çıkmaz (python ile denendi). Mod ile çalışılırken bütün işlemler $\bmod Q$ yapılır; çıkarma sonucu negatif olabileceği için Q eklenerek pozitife çekilir (bu not için yazılıp doğrulanan C programında böyle yapıldı). Hoca bu kısmı anlatırken bir yerde "31478'den 14782'ye geçerken" dedi; kastedilen, ilk pencereden ikinciye geçiştir.

11.3 Bit Kaydırma ile Yapılabilir mi?

Soru: Dersin sonunda bir öğrenci, baştaki 3'ü düşürmek için sola ya da sağa kaydırma (shift) operatörü kullanılsa yanlış olur mu diye sordu.

Cevap: Hoca, kaydırma operatörünün ikilik tabanda (bitler üzerinde) çalıştığını hatırlattı. 8 sayısı bir kez sağa kaydırılırsa 4 bulunur; sağa bir kaydırma ikiye bölmektir. 31415 gibi bir sayı sağa kaydırılırsa ikilik düzende bir kaydırma yapılır, yani sayı ikiye bölünür; oysa burada onluk düzende basamak düşürmek istenmektedir. Bu yüzden onluk tabanda kaydırma operatörü işe yaramaz. Taban $B = 2$ olsaydı öğrencinin dediği doğru çalışırdı.

12. Üç Yöntemin Karşılaştırması

Hoca dersi üç yöntemin karmaşıklığını karşılaştırarak bitirdi:

- **Kaba kuvvet:** Her uyumsuzlukta en başa dönüldüğü için hoca "$M \cdot N$'den daha kısa olamıyor" dedi. Kaba kuvveti kısaltmanın, küçültmenin imkanı yoktur.
- **Boyer-Moore:** Ortalama durumda yaklaşık N/M.
- **Rabin-Karp:** Tipik durumda karmaşıklığı yaklaşık N'dir; M'ye çok bağlı değildir, sabit bir çarpanla N'dir. Hoca, bu sabitin olasılık hesabıyla yaklaşık $7N$ gibi bir değer olarak bulunduğunu söyledi.

Yöntem	Tipik durum	En kötü durum
Kaba kuvvet	yaklaşık N (kısa örüntüde)	$(N - M + 1) \cdot M \approx MN$
Boyer-Moore	yaklaşık N/M	MN

Yöntem	Tipik durum	En kötü durum
Rabin-Karp	yaklaşık $7N$	MN (her özet eşitliği karakter karakter doğrulanırsa)

Hocanın mesajı şudur: Boyer-Moore ve Rabin-Karp'ta karşılaştırma sayısının MN'den çok daha kısa olma ihtimali yüksektir.

> **Ders sonrası not:** Hoca kaba kuvvetin "her zaman $M \cdot N$ olmak zorunda" olduğunu söyledi. $M \cdot N$ en kötü durumdur; dersin başında hocanın kendisi de en iyi durumu N olarak vermişti. Gerçek metinlerde uyumsuzluk genelde ilk bir iki karakterde çıktığı için kaba kuvvet tipik olarak büyük olasılıkla yaklaşık $1{,}1N$ karşılaştırma yapar; sorunu, en kötü durumun garanti altında olmamasıdır. Hoca Rabin-Karp'ın en kötü durumu için "M çarpı M" dedi. Özetler her eşitlikte karakter karakter doğrulanırsa ve çok sayıda çakışma ya da eşleşme olursa en kötü durum $M \cdot N$'dir.
>
> **Ek bilgi (derste anlatılmadı):** Rabin-Karp'ın iki sürümü vardır. Özet eşitliğini karakter karakter doğrulamayan sürüme Monte Carlo sürümü denir; her zaman doğrusal sürede çalışır ama çok küçük bir olasılıkla yanlış sonuç verebilir. Doğrulama yapan sürüme Las Vegas sürümü denir; sonucu her zaman doğrudur ama en kötü durumda MN sürebilir. Derste anlatılan, eşitlikte doğrulama yapan sürümdür.
>
> **Ek bilgi (derste anlatılmadı):** Uyumsuz karaktere göre kaydırma kuralına "kötü karakter kuralı (bad character rule)" denir. Bu kural büyük olasılıkla 1977'de yayımlanan özgün Boyer-Moore algoritmasının bir parçasıdır; özgün algoritmada ayrıca "iyi sonek (good suffix)" kuralı da vardır. Horspool'un katkısı büyük olasılıkla 1980'de bu kuralı tek bir tabloyla kullanan basitleştirilmiş sürümdür. Derste anlatılan sürüm, yalnız kötü karakter tablosunu kullanan bu basitleştirilmiş Boyer-Moore'dur.

Örnek Sorular

Bu derste hazır ekran karesi ve video yoktur; aşağıdaki soruların hepsi derste sözlü olarak sorulmuş ya da tahtada hocanın sınıfa yönelttiği sorulardır.

Soru 1 (derste sözlü, 00:26:34): Kaba kuvvet arama fonksiyonunda `for` döngüsünün içinde `return i` denirse, bu `break` gibi döngüyü kırıp mı çıkar?

> **Çözüm:** `return` döngüyü değil fonksiyonu sonlandırır; fonksiyondan çıkıldığı için döngü de biter. Hoca okunabilirlik için aynı algoritmayı `while ((i <= n - m) && (!found))` koşuluyla ve `found` bayrağıyla yazmayı önerdi (3.1'deki `search2`). İki sürüm `gcc -Wall` ile derlenip aynı sonuçları verdi.

Soru 2 (derste sözlü, 00:41:14): Boyer-Moore'da uyumsuz karakter örüntü içinde birkaç kez geçiyorsa (`NEEDLE`'daki üç `E` gibi), örüntü hangi geçişle hizalanır?

> **Çözüm:** Karakterin örüntüdeki en sağdaki geçişiyle hizalanır. Böylece en küçük güvenli kaydırma yapılır ve hiçbir olası eşleşme atlanmaz. `NEEDLE` için `E`'nin tablo değeri 5'tir.

Soru 3 (derste sözlü, 01:02:49): `SOPA` örüntüsü metinde $i = 5$'e oturtulmuş; metnin 5-9. karakterleri `KÖPAK`. `A` ve `P` uyuyor, $j = 1$'de `Ö` ile `O` uymuyor. Yola nereden devam edilmelidir?

> **Çözüm:** `Ö` örüntüde yoktur (durum 1). $i = i + j + 1 = 5 + 1 + 1 = 7$. Arama `Ö`'den sonraki karakterden, yani `PAK`'tan devam eder. Tablo formülüyle de skip $= 1 - (-1) = 2$ çıkar (python ile doğrulandı).

Soru 4 (derste sözlü, 01:13:38): Metinde 9-14. konumlarda `SOKARA` yazıyor. `KARA` $i = 9$'a oturtulmuş. $j = 3$'te `A`'lar uyuyor, $j = 2$'de metindeki `K` örüntünün `R`'si ile uymuyor. `i` nereden başlatılmalıdır?

> **Çözüm:** `K` örüntüde 0. gözdedir (durum 2a). $i = 9 + 2 - 0 = 11$. Bu konumda örüntünün `K`'sı metnin `K`'sı ile çakışır ve `KARA` bulunur (python ile doğrulandı).

Soru 5 (derste sözlü, 01:34:10): `shiftLookUp` fonksiyonundaki `offset` her zaman `'A'` mıdır?

> **Çözüm:** Hayır. Büyük harf tablosu için `'A'`, küçük harf tablosu için `'a'` alınır; başka sembollerde ona göre seçilir. Bütün ASCII tablosu kullanılırsa offset gerekmez, karakter kodu doğrudan indistir.

Soru 6 (derste sözlü, 01:46:26): `NEEDLE` $i = 9$'a oturtulmuş, $j = 5$'te `E`'ler uyuyor, $j = 4$'te metinde yine bir `E` var ve örüntünün `L`'si ile uymuyor. Yeni `i` ne olur?

> **Çözüm:** `E`'nin tablo değeri 5'tir. $i + j - 5 = 9 + 4 - 5 = 8 < 9$; kaydırma $4 - 5 = -1$ negatiftir (durum 2b). Geri gidilmez, $i = 10$ yapılır.

Soru 7 (derste sözlü, 02:13:17): Rabin-Karp'ta hatalı sonuç olma ihtimali yok mu? Örneğin 997'nin 2 katının 613 fazlası ile 3 katının 613 fazlası olan iki sayının modu aynı çıkar.

> **Çözüm:** Evet, farklı sayıların modu aynı olabilir (çakışma). Bu yüzden özetler eşit çıktığında karakterler tek tek karşılaştırılır; yalnız bu doğrulama da geçerse örüntü bulunmuş sayılır. Çakışmayı azaltmak için Q büyük bir asal seçilir, ama çakışma tamamen ortadan kalkmaz.

Soru 8 (derste sözlü, 02:16:53): Özetler eşit çıkınca karakter karakter karşılaştırmak yerine, 997'ye bölmede çıkan bölümler de saklanıp karşılaştırılamaz mı?

> **Çözüm:** Hoca, kalanı aynı olan iki farklı sayının bölümleri farklı olacağı için bu yolla da ayırt edilebileceğini kabul etti. Derste anlatılan Rabin-Karp yöntemi ise bu noktada karakterleri tek tek karşılaştırır.

Soru 9 (derste sözlü, 02:18:48): 31415, 14159, 41592 gibi her pencerenin sayısı basamaklardan baştan oluşturuluyor. Bu hesap nasıl kısaltılabilir?

> **Çözüm:** Bir önceki pencerenin değeri kullanılır: baştaki basamağın katkısı çıkarılır, kalan B ile çarpılır ve yeni basamak eklenir: $x_{i+1} = (x_i - t_i \cdot B^{M-1}) \cdot B + t_{i+M}$. Örnek: $(31415 - 3 \cdot 10^4) \cdot 10 + 9 = 14159$. Mod ile: $14159 \bmod 997 = 201$ (python ve C ile doğrulandı).

Soru 10 (derste sözlü, 02:28:46): Baştaki basamağı düşürmek için sağa ya da sola bit kaydırma operatörü kullanılabilir mi?

> **Çözüm:** Kaydırma operatörü ikilik tabanda çalışır; sağa bir kaydırma ikiye bölmektir ($8 \to 4$). Onluk tabanda basamak düşürmek için işe yaramaz. Taban $B = 2$ olsaydı kaydırma doğru çalışırdı.

Dersin Toparlaması

Bu derste, bir metin içinde M uzunluklu bir örüntüyü aramak için üç yöntem görüldü. Kaba kuvvet, örüntüyü her konuma oturtup soldan sağa karşılaştırır; uyumsuzlukta bir karakter kayar ve baştan başlar. En iyi durumda yaklaşık N, en kötü durumda $(N - M + 1) \cdot M$ karşılaştırma yapar; kısa örüntülerde sorun çıkarmaz, uzun örüntülerde $O(N^2)$'ye yaklaşabilir. Boyer-Moore, örüntüyü sondan başa karşılaştırır ve uyumsuz karakterin örüntüdeki en sağdaki yerine göre kaydırır. Örüntü değişmediği için bu kaydırma bilgisi `rightSkip` tablosunda önceden hazırlanır; böylece tipik durumda yaklaşık N/M karşılaştırma yapılır, en kötü durum yine MN'dir.

Rabin-Karp ise karakterleri tek tek karşılaştırmak yerine örüntünün ve her pencerenin büyük bir asala göre modunu karşılaştırır. Çakışma ihtimali yüzünden özetler eşit çıktığında karakterler doğrulanır. Her pencerenin sayısı bir önceki pencereden kayan özet formülüyle sabit sürede bulunur. Hoca, sayısız yöntem arasından yalnız basit bir iki tanesinin işlendiğini söyledi; böl ve yönet sıralama algoritmalarının sonraki yıl Algoritma Analizi dersinde görüleceğini de dersin başında belirtmişti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.